Table des matières

Labo 06 — Remonter le temps, cloner, partager 1
Prérequis 1
Étape 0 — Poser le contexte 1
Étape 1 — Le Time Travel (annuler une erreur) 1
Étape 2 — Le clonage zero-copy 2
Étape 3 — Rétention : Time Travel et Fail-safe 3
Étape 4 — Créer un share (côté fournisseur) 3
Nettoyage 3
Récapitulatif 4

Labo 06 — Remonter le temps, cloner, partager

Module 6/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

But du labo — *Voir* le domaine 5 : récupérer une table avec le Time Travel (et `UNDROP`), créer un clone zero-copy et le faire diverger de l'original, lire la rétention (Time Travel / Fail-safe), puis monter un share côté fournisseur. Comptez 25 à 35 minutes. Tout se fait dans un worksheet Snowsight.

Prérequis

- Le Cours 00 est fait : entrepôt `SNOWPRO_WH` + base `SNOWPRO_DB.SANDBOX`.
- L'étape 4 (partage) crée un share côté fournisseur ; le consommer demanderait un 2ᵉ compte, on s'arrête donc à la création, qui suffit à voir le mécanisme.

Étape 0 — Poser le contexte

```
USE ROLE SYSADMIN;
USE WAREHOUSE SNOWPRO_WH;
USE DATABASE SNOWPRO_DB;
USE SCHEMA SANDBOX;
```

Étape 1 — Le Time Travel (annuler une erreur)

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_06_SNOW.md`, section 4.

```
CREATE OR REPLACE TABLE journal (id INT, v STRING);
INSERT INTO journal VALUES (1,'a'), (2,'b'), (3,'c');
SELECT COUNT(*) FROM journal;                  -- 3

-- Oups : on vide la table par erreur.
DELETE FROM journal;
SET del_id = LAST_QUERY_ID();                  -- on retient l'ID du DELETE
SELECT COUNT(*) FROM journal;                  -- 0

-- Time Travel : relire l'état JUSTE AVANT le DELETE.
SELECT COUNT(*) FROM journal BEFORE(STATEMENT => $del_id);   -- 3 !

-- Restaurer pour de vrai : réinsérer depuis ce passé.
INSERT INTO journal SELECT * FROM journal BEFORE(STATEMENT => $del_id);
SELECT COUNT(*) FROM journal;                  -- 3, récupérées

-- Et si on supprime la table entière ? UNDROP la ressuscite.
DROP TABLE journal;
UNDROP TABLE journal;
SELECT COUNT(*) FROM journal;                  -- toujours là
```

Ce que vous observez. `BEFORE(STATEMENT => …)` lit la table telle qu'elle était avant une requête donnée, même après un `DELETE`. `UNDROP` ressuscite une table supprimée. Le Time Travel est votre filet contre les fausses manœuvres, dans la fenêtre de rétention.

Étape 2 — Le clonage zero-copy

Théorie : `Theorie_06_SNOW.md`, section 3.

```
-- Un clone INSTANTANÉ (aucune donnée copiée : mêmes micro-partitions).
CREATE TABLE journal_bac CLONE journal;

-- On modifie le CLONE...
INSERT INTO journal_bac VALUES (4,'d'), (5,'e');

-- ... l'original n'a pas bougé (copy-on-write).
SELECT COUNT(*) FROM journal;       -- 3 (intact)
SELECT COUNT(*) FROM journal_bac;   -- 5 (le clone a divergé)
```

Ce que vous observez. Le `CLONE` est immédiat, même sur une grosse table, parce qu'il ne copie rien au départ : il pointe vers les mêmes partitions. Dès que vous écrivez, seules les partitions changées sont réécrites, et les deux tables divergent sans se gêner. C'est l'outil rêvé pour un bac à sable sur des données de prod.

© 2026 iAlexMG. Tous droits réservés.

Étape 3 — Rétention : Time Travel et Fail-safe

Théorie : `Theorie_06_SNOW.md`, sections 4 et 5.

```
-- La fenêtre de Time Travel de la table (en jours).
SHOW PARAMETERS LIKE 'DATA_RETENTION_TIME_IN_DAYS' IN TABLE journal;

-- On peut l'ajuster (0 à 1 en Standard ; jusqu'à 90 en Enterprise+).
ALTER TABLE journal SET DATA_RETENTION_TIME_IN_DAYS = 1;
```

Ce que vous observez. `DATA_RETENTION_TIME_IN_DAYS` fixe la profondeur du Time Travel. Le Fail-safe, lui, n'apparaît pas comme un réglage : ce sont 7 jours fixes, après le Time Travel, récupérables uniquement par le support Snowflake, et seulement pour les tables permanentes (les `transient` et les temporaires n'en ont pas). On ne le manipule pas, on le connaît.

Étape 4 — Créer un share (côté fournisseur)

Théorie : `Theorie_06_SNOW.md`, section 2.

```
USE ROLE ACCOUNTADMIN;                          -- le partage relève de l'admin du compte

CREATE OR REPLACE SHARE partage_demo;

-- On accorde au SHARE le droit de traverser les conteneurs, puis de lire la table.
GRANT USAGE  ON DATABASE SNOWPRO_DB          TO SHARE partage_demo;
GRANT USAGE  ON SCHEMA   SNOWPRO_DB.SANDBOX  TO SHARE partage_demo;
GRANT SELECT ON TABLE    SNOWPRO_DB.SANDBOX.journal TO SHARE partage_demo;

SHOW SHARES;                                    -- votre share apparaît, type OUTBOUND
USE ROLE SYSADMIN;
```

Ce que vous observez. Vous venez de préparer un partage sortant (*outbound*) : un consommateur qui le monterait lirait la table `journal` live, sans copie. On s'arrête ici, car le consommer exigerait un 2e compte (ou un *reader account*). Le mécanisme, lui, est en place.

Nettoyage

```
USE ROLE ACCOUNTADMIN;
DROP SHARE IF EXISTS partage_demo;
USE ROLE SYSADMIN;
DROP TABLE IF EXISTS journal_bac;
DROP TABLE IF EXISTS journal;
```


© 2026 iAlexMG. Tous droits réservés.

Récapitulatif

- ☐ J'ai récupéré des données avec le Time Travel (`BEFORE`) et `UNDROP`.
- ☐ J'ai créé un clone zero-copy et l'ai fait diverger de l'original.
- ☐ J'ai lu la rétention (`DATA_RETENTION_TIME_IN_DAYS`) et je distingue Time Travel de Fail-safe.
- ☐ J'ai monté un share côté fournisseur (`GRANT … TO SHARE`).

Tout coché, vous tenez le domaine de la collaboration. Cap sur le Cours 07, la révision pondérée et l'examen blanc.

© 2026 iAlexMG. Tous droits réservés.